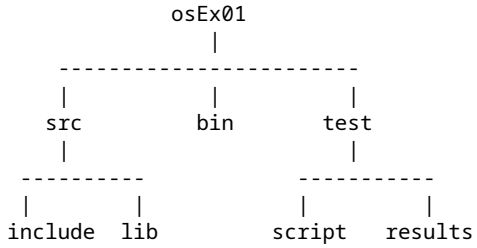
Operating Systems Course: Lab01

Laboratory number 01

Exercise 01: Management of files and directories

Part 01

Produce the following tree of directories in your home directory:



Part 02

- Download the file: lab01e01in.txt [https://www.skenz.it/listing/os/lab/lab01e01in.txt]
- Copy the file in the directory `src`, then, using the directory `src` as the current directory, copy the file in the directories `test`, `script`, and `results` (to this extent use only absolute paths).
- Check the existence of the files previously created, and check their rights.
- Use the commands `more`, `less` and `cat` to verify the content of the files.
- Use the command `diff` to verify that all the files have the same content.
- Without changing the current (working) directory, cancel using relative paths all the files previously created.
- Repeat the same sequence of operations starting from father directory of `src`, i.e., `osEx01`.

Part 03

Executing all the commands from the root of your user (i.e., `/home/username`), do the following:

- Copy all the directory tree named `osEx01` into a directory tree named `osEx01backup`, by using the command `cp` to copy individual files and the command `mkdir` for directories.
- Cancel all the content of `osEx01` by using the command `rm` to remove individual files and the command `rmdir` for directories“.
- Repeat both the *copy* and the *cancel* operations by using the *recursive* versions of the commands `cp` and `rm`.

Part 04

Check what they do and for what the following commands are used:

- `pwd`
- `wc`

- `history`

For any problem, remember that you can use the following command to obtain an online help manual for a specific `command`:

- `man <command>`

Exercise 02: Permissions management

Verify and reply to the following questions:

- the `username` and the `group` to which you belong.
  Is it possible to modify them?
- the position of your home directory within the file system hierarchy of the system you are using
- the structure of your `/home` directory (e.g., its subdirectories, etc.)
- the rights of reading and writing on the various directories of the file system. Is it possible to modify access rights to your `home` directory?
- what happens if *reading* or *execution* rights are eliminated for a directory. Use both the octal notation (e.g., “654”) and the one based on characters (e.g., ”+uw-gx“) to modify the rights.

How is it possible to modify in the same way the access rights to **all** the files and directories of a directory tree (for example, to eliminate the *read* right to all, user, group, and other of the directory `osEx01backup`)?

Esercise 03: Link and comparisons between files

- Download the file lab01e03in.txt [https://www.skenz.it/listing/os/lab/lab01e03in.txt]
- In the same directory where the file is stored, you have to create:
  - A copy of the file with the name `lab01e03in.copy`
  - A hard-link to the file with name `lab01e03in.hl`
  - A soft-link to the file with name `lab01e03in.sl`
- View files information with the command `ls -l`
  - What can you see in the second column of the output?
  - Why don't all the files have the same size?
- Check that the content of the three created files is the same as the original file using the `diff` command.
  - Why does the `lab01e03in.sl` file have a different size than the others, but it has the same content?
- Open and modify the content of the file `lab01e03in.txt` with a text editor of your choice
  - How do you expect information related to the size and the date of last modification of the file will be changed? Why?
  - Verify your answer using the command `ls -l`
  - What changes are expected regarding the content of the files? Why?
  - Check the response by comparing the three files created with the original one. Use again the `diff` command.
- Rename the file `lab01e03in.txt` in `lab01e03in.backup`
  - What can you see now from the output of the `ls -l` command?
  - Compare the content of the file `lab01e03in.backup` with the content of the files `lab01e03in.hl` and `lab01e03in.sl`. Why in the second case do you obtain an error message?
- Create a new empty file named `lab01e03in.txt`
  - What is the current expected content of the file `lab01e03in.sl`? Why?
  - Check with the command `cat`

Observation:
The command
`touch <nomefile>`
allows to update date and time of the last modification performed on a file.
In the case the specified file does not exist, it creates an empty file with that name.



Exercise 04: C programs and compilation

Perform the following tasks using:

- an editor of your choice (e.g., `vim`, `emacs`, `gedit`, etc.)
- the `gcc` compiler (alternatively use an IDE of your choice, e.g., CodeBlocks).

Part 01

Write a C language program that, using the ANSI C I/O primitives (`fgetc`/`fputc`, `fscanf`/`fprint`,`fget`/`fputs` of your choice), is able to copy a text file to an identical file.
The name of the two files is received on the command line, so that the program can be executed using a command that specifies three parameters:
`nameExcutable sourceFile destinationFile`

Part 02

Compile the source files using `gcc`

- Eliminate possible warnings.
- Try to compile the file by directly generating the executable, or first the object code and then the executable.
- Run the program using the lab01e01in.txt [https://www.skenz.it/listing/os/lab/lab01e01in.txt] file as a test file.
- Verify the program using as program arguments both *relative paths* and *absolute paths* to specify input and output files.
- Verify the correct copy of the files (i.e., `sourceFile` and `destinationFile` must be identic) using the shell command `diff`.

Part 03

How does the written program behave if you try to copy a binary file?

- Use the object file or the executable file of the program written to perform a check.
- Test its correct functioning using the `diff` command.

Part 04

Change the program using

- ANSI C `fread` and `fwrite` functions (optional) and then
- `open`, `read`, `write`, and `close` POSIX system calls.
- Test its correct functioning using the `diff` command.

Exercise 05: Compilation and makefile

- Compile the program of the previous exercise using a Makefile containing only the compilation target.
- Edit the previous Makefile by adding the `install` target, which allows to
    1. create a directory, sibling of the work directory and named `bin`
    2. move the created executable in this directory
- Edit the previous Makefile by adding the `clean` target, which removes the object file in the current directory
- Edit the previous Makefile by adding the `distclean` target that calls the `clean` target and deletes the `bin` directory (which is the sibling of the current directory) and all its content
- Edit the previous Makefile using variables to specify the compiler, the compilation options, and the file name.

Take as reference the example analyzed in classroom: Makefile4 [https://www.skenz.it/listing/os/u02-Linux-environment/u02s02e/01-compiler/Makefile4]

If you found any error, or if you want to partecipate to the editing of this wiki, please contact: admin [at] skenz.it

You can reuse, distribute or modify the content of this page, but you must cite in any document (or webpage) this

url: https://www.skenz.it/os/lab01